



C-STAR

Final Report Document

Primary Contact

Nathan Cao

Project Team

Michelle Freeman - Project Lead

Kaden Du - Electrical Technical Lead

Tom Eckel - Mechanical Technical Lead

Neha Joby - Electrical Engineer

Amberley Martinez - Mechanical Engineer

Susan Badanai - Electrical Engineer

Jason Widjaja - Mechanical Engineer

Catherine Menten - Electrical Engineer

Sylvain MacGovern - Mechanical Engineer

Katrina Le - Electrical Engineer

Diana Sooknauth - Mechanical Engineer

Vasishta Malisetty - Electrical Engineer

Elyssa Grube - Industrial Designer



TABLE OF CONTENTS

Goal	2
General Guidelines	2
Executive Summary	3
Purpose	3
Background Research	4
Systems Overview	5
Mechanical Design	6
Assembly Overview	6
Name of Component	6
Hardware Selection	7
Alternative Designs	7
Electrical Design	8
Design Overview	8
Sensors	8
Circuit Board	8
Software Design	9
General Software Design	9
Explanation of Overall Software Decisions	9
Communication	9
Next Steps	10
Appendix	11



Executive Summary

Purpose

This project is aiming to automate the manual & tedious repair estimate process for parking garages. Structural engineering firms must perform assessments for each new project to understand what areas of concrete must be repaired and estimate the cost and labor required. The client has requested a product that will reduce the manual burden on structural engineers while simultaneously improving the efficiency and accuracy of repair assessments. Since most delaminations can't be visually identified, engineers currently use impact sounding techniques such as dragging chains across the floor or tapping the concrete with hammers, and then listen to the sounds produced. Trained engineers can differentiate between sound and delaminated concrete and mark the problem areas on a floor plan. However, there are many opportunities for human error and bias to impact repair assessments. For example, engineers can get tired after many hours of sounding and become less discerning about whether or not a section of concrete is delaminated. These manual techniques are time consuming and the potential for errors can hinder cost analysis and repair timelines. The proposed solution is a semi-autonomous robot that will traverse a section of concrete, perform impact sounding as it moves, and determine the location of delaminations. This robot will only require minimal setup by an engineer prior to use, including uploading a floor plan CAD file to a bluetooth-connected computer, which will increase the accuracy of repair assessments while greatly reducing the time and manual labor required. Once the robot is operating, the user will only need to perform periodic monitoring of the device, with less interaction needed as device development progresses.



The reader should understand what problem the venture is trying to solve and what Generate did to address this problem over the semester.

- Similar to a Lab Report's Abstract
- Aim for 250 words
- Communicate what problem the client is solving
- Communicate what steps Generate helped the product take
- This can be further research on the problem, prototyping to test a solution, creating foundations for the product, etc.
- Communicate how Generate planned to achieve that goal at a high level
- Touch on learning opportunities within the team

Michelle Freeman

Project Lead

freeman.mi@northeastern.edu



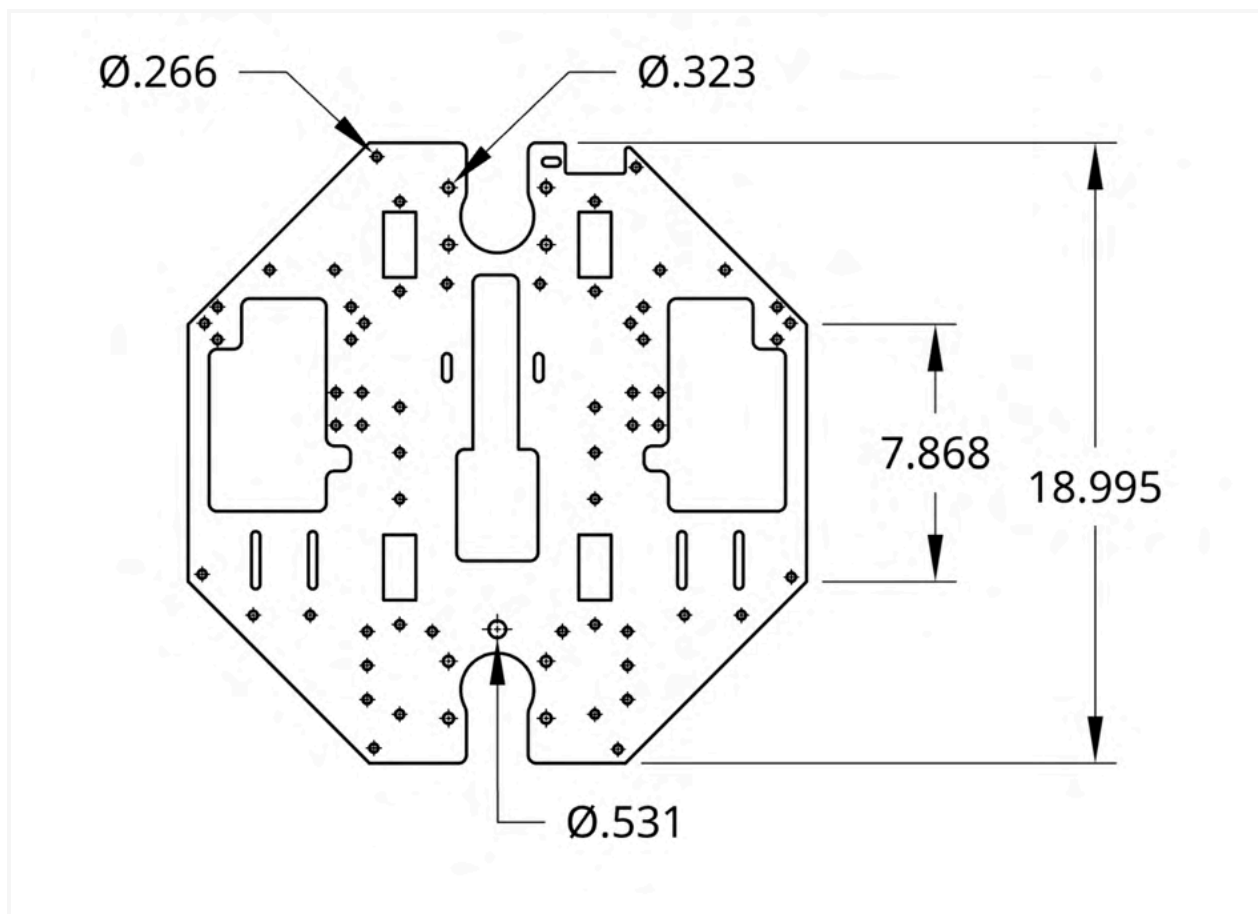
Background Research

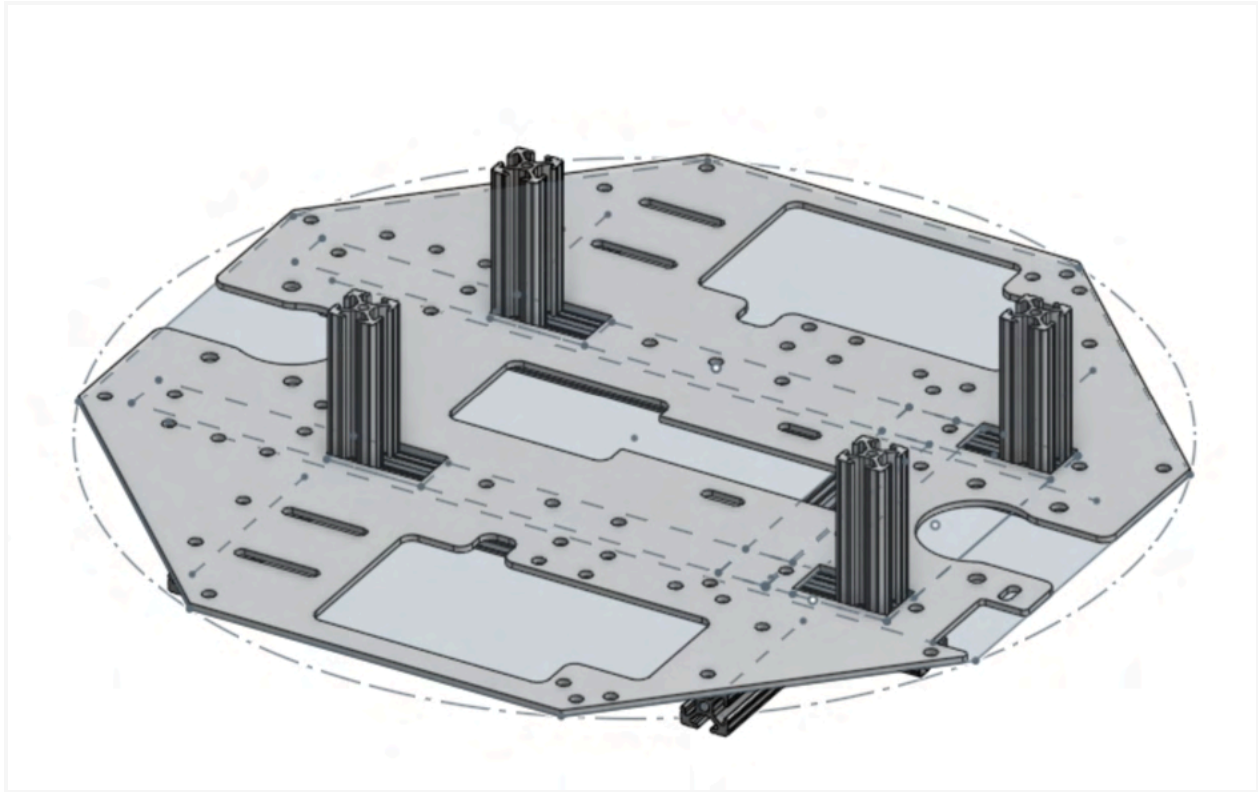
There are no direct competitors for this product, as many engineering firms are still using manual impact sounding methods. [Research](#) was conducted in the 1970s on creating a push-cart type device that will record impact sounding and determine delamination, but this was never commercialized and still required manual input from the user. There are some products ([1](#), [2](#)) that use radar and imaging methods to detect rebar, electric cables, and other objects embedded within concrete. Other [products](#), used primarily for bridges, use cameras and ultrasonic waves to detect unsound structures. These are more suited for larger areas, since structures such as parking garages require devices with a smaller footprint. Also, these devices use extremely expensive equipment, while the proposed device is significantly more economical. So, there are no semi-autonomous products on the market that use impact sounding methods to analyze concrete structures.

Mechanical Design

Chassis

The chassis serves to connect and support all mechanical, electrical, and housing components in the most efficient configuration to best fulfill their purposes. The chassis consists of a base plate and 80/20 framing. Holes, slots, and cutouts were planned around the main components, including the drive train, the sounding tool, ultrasonic sensors, and other holes for connective brackets.





The design's function is to connect and support all mechanical, electrical, aesthetic components while maintaining the structural integrity and staying within the weight constraint of 35 pounds.

Regarding material selection, Aluminum 6061 is the highest cost-effective to strength ratio for the final prototype. The 80/20 framing was used as support due to its durability, assembly readiness, and universality. Important tolerances to keep in mind include the clearance holes for the 80/20 connections ($\frac{1}{4}$ –20 holes), the drive train, sound tool, caster wheel, and bracket holes. A hole was added to the base plate last minute using a $\frac{1}{2}$ " bit drill that was used to route the USB-C cable through.

Designing for Assembly (DFA) ideas to keep in mind would be to change the 80/20 corner brackets to L brackets so the 80/20 can be connected from the outside. The corner bracket made it hard to reach under the robot during assembly, so using the L shaped brackets would have been helpful to make assembly easier, especially when integrating the electrical components. Other DFA constraints include ensuring all cutouts are filleted and have significant tolerance in order to fit the components well, especially the sounding tool, drivetrain, and the 80/20 uprights. The slots for the uprights should have at least 0.05" of tolerance on both sides,



and any additional holes/components must be at least 0.5" away from the edge of the upright slot in order to account for the t-slot nuts.

The second floor where the battery and PCB components are housed was originally designed to be another plate. However, considering the significant weight of the battery (6 lbs), it was decided to move the battery closer to the middle and attach it to the 80/20 uprights to ensure the center of mass was closer to the middle of the robot. Furthermore, the PCB was vertically fastened to the 80/20 uprights using 3D printed brackets. Its orientation was optimized to best attach cables to components and the PCB.

Manufacturing recommendations for the base plate prototyping include using Medium-Density Fiberboard (MDF) or High-Density Fiberboard (HDF) with the dimensions 24"x 48", laser cutting on a bed of the same dimensions (use EXP 2'x4'). For the final prototype, the chassis base plate was sent to Send-Cut-Send using Aluminum 6061, at 0.125" thick. The cost of the base plate was \$__ for 2 plates to be cut. The cost of the 80/20 was \$__ for 10 feet, in which we used ~__" out of 120".

Future state recommendations and CAD optimization include more slots and/or holes in the base plate for cable routing and management, as well as furthering consolidation of fastener type (i.e. mostly ¼-20 for base plate connections).

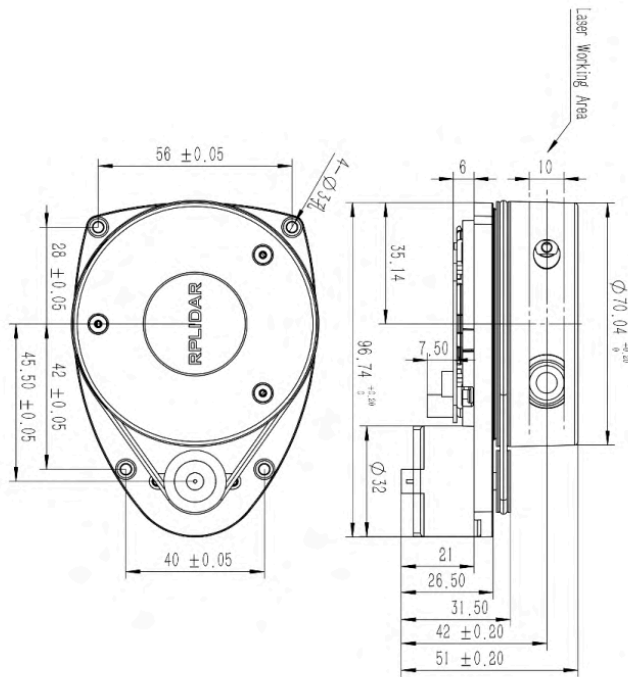
LiDAR

The LiDAR sensor served as the "eyes" for the robot, allowing the robot to detect any obstacles in its environment. Obstacle detection is important for the robot

The LiDAR sensor we decided to use is the RPLIDAR A1M8-R6 by DF Robot. This LiDAR is a 360 degree sensor, with a 12 meter radial range. Another benefit to this specific LiDAR is the fact that a motor was included in the kit, which gave us one less expense and also made the subsystem more compact.



Figure X: RPLiDAR A1M8-R6 - 360 Degree LiDAR Laser Range Scanner (12m), sourced from www.dfrobot.com



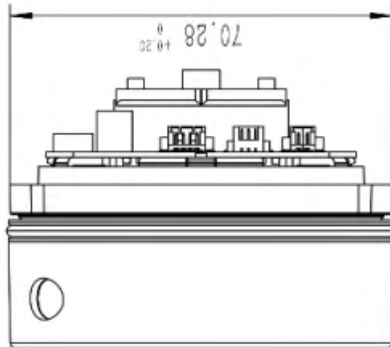


Figure X: Mechanical Dimensions of RPLiDAR A1M8-R6 , sourced from the datasheet provided by www.dfrobot.com

In the future, an upgrade to the LiDAR subsystem could be implementing a LiDAR with a longer radial range. The reason we were not able to implement this was because of our budget constraints; LiDARs with longer radial range increase in cost.

Ultrasonic Bracket Mounting

We decided to create custom brackets to mount the ultrasonic sensors because of convenience and flexibility when deciding where to place them. We had four ultrasonic sensors in total- one pointing downwards and three placed parallel to the lower panels of the robot. The ultrasonic sensor that was pointing downwards, the ledge detector bracket, ended up having to be merged with one of the lower corner brackets that were being used to hold two panels together. This was done to prevent crowding of parts on the chassis.

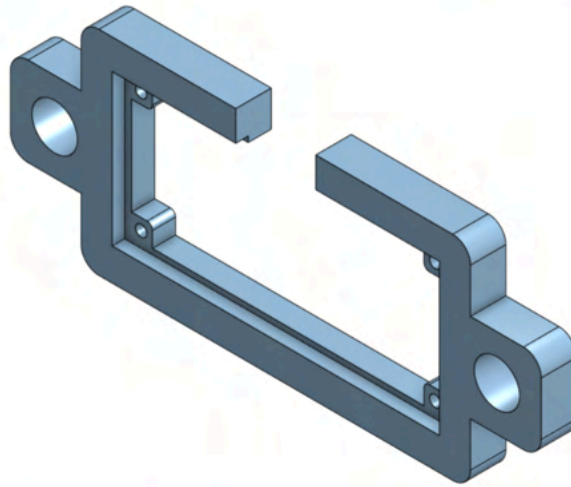


Figure X: Isometric View of Custom Ultrasonic Brackets

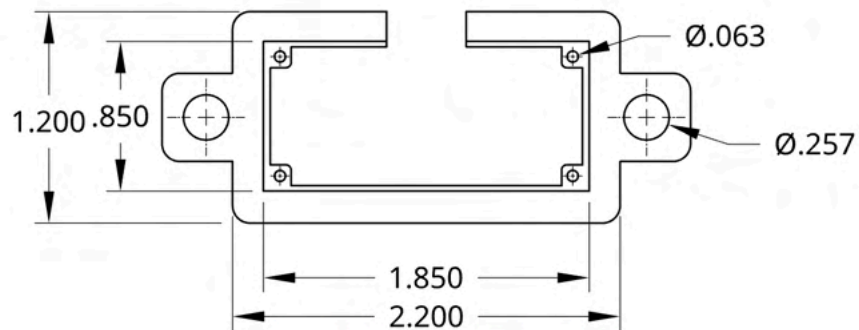


Figure X: Basic Dimensions of Custom Ultrasonic Brackets

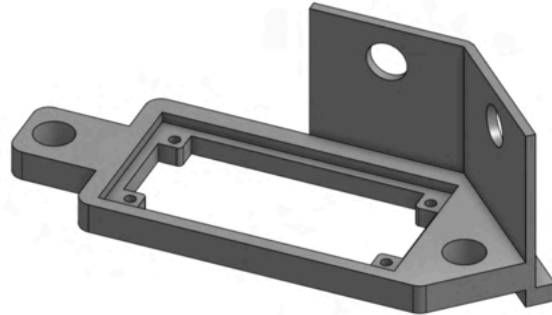


Figure X: Isometric View of Custom Ledge Detector Bracket

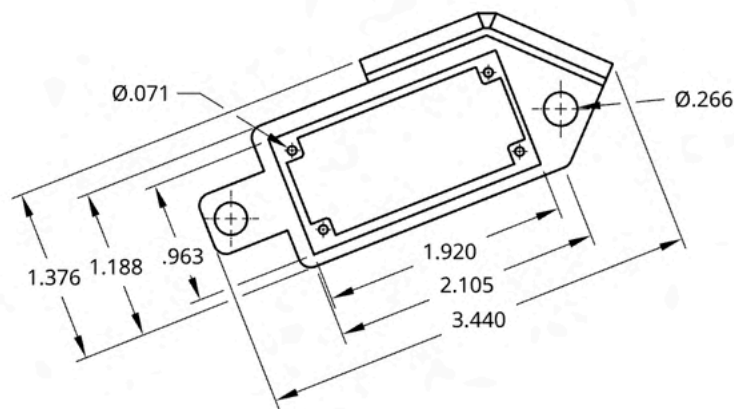


Figure X: Basic Dimensions of Custom Ledge Detector Bracket

Out of convenience, these brackets were printed using a Objet30 3D Printer which uses VeroClear, a material that is similar to acrylic. Using this printer and material allowed us to have high precision with the final results of the parts, which was beneficial considering the small dimensions we were working with.

The brackets were both designed so that the electronics on both sides of the ultrasonic sensor were easily accessible. It is also important to note that the mounting holes on the side of the brackets that were being placed against a surface were counterbore holes, so that the mounting screws would sit flush against the surface.

Sounder Mechanism

The Sounder Mechanism serves to determine delaminated concrete for core infrastructures ranging from parking garages to buildings and many more. One sprocket was used to project noise to a microphone that will be signally processed through an ADC (Analog to Digital Converter). These sprockets were passively driven by the entire robot and were placed within a noise-proof structure to prevent any sources of noise pollution. A 3D-printed linkage was used to connect to the ceiling of the chassis structure via a shaft and bearing, allowing for free rotation of one of the ends. This gives some flexibility in terms of possible speed bumps that may propagate greater concentrated stresses around the sprocket. The sounder mechanism was eventually tested in the garage and results proved that it sounded quieter than expected. Several options can be done to optimize this in the future. Some of the options include making the sprockets thicker, adding more sprockets in parallel, adding more teeth to the sprockets, and altering the tooth profile of the sprocket to be sharper to enhance the sound of the sprocket on concrete.

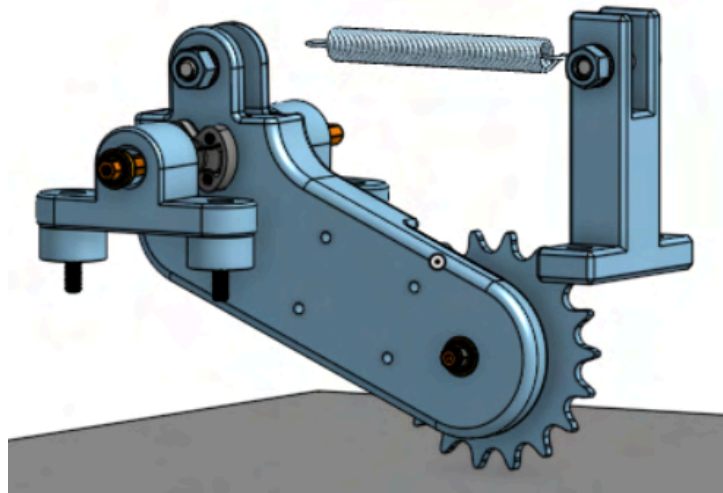


Figure #: Sounder Mechanism Assembly

The initial design of the Sounder Mechanism consisted of two sprockets laid with two linkages with four vibration dampers that were fastened to the chassis base plate. However, this design added unnecessary complexity as these two linkages posed additional risks when the sprocket was approached by hurdles such as speed bumps. Furthermore, due to space constraints, it proved challenging to incorporate 2 sprockets in the design.

Sounder Linkage

This 3D-printed Sounder Linkage serve as the primary component responsible for integrating all elements of the Sounder Mechanism. Connected via an 8mm hex shaft to damping mounts, its

role is crucial in mitigating vibrations from transmitting from the sprocket to the entirety of the enclosed 80-20 structure. The accompanying figure illustrates the various stages of evolution that the sounder linkage underwent to reach its current prototype. It underwent modifications, including size reduction to fit within the sheet metal base plate, with further dimensional adjustments made to accommodate the placement of the mic PCB in close proximity to the sprocket.



Figure #: Sounder Linkage Iteration

Vibration Mounts

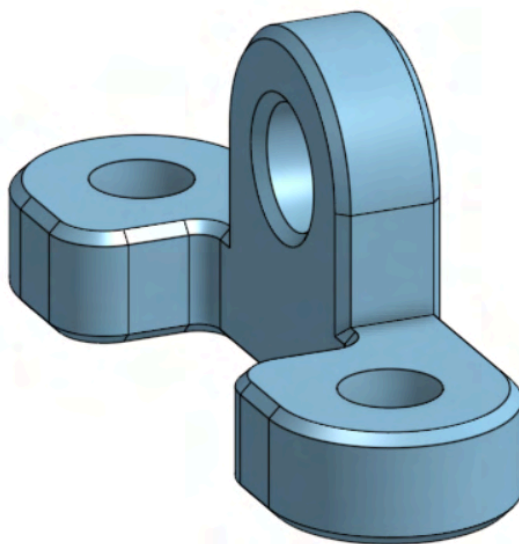


Figure #: Vibration Mounts

These 3D-printed vibration mounts are secured to the chassis using two M6 x 10 mm socket head screws, which were affixed to two vibration-damping mounts sourced from McMaster. These damping mounts were then directly fastened to the chassis using their integrated stud and insert. To secure these mounts to the base plate structure, an M6 nut was employed. The central aperture accommodates an 8mm hex bearing, crucial for the linkage to the sounder. Additionally, a cut-out was implemented on the vibration mount to preempt any potential interference between the mount and the Sonic Hub, directly linked to the linkage.

Spring Mount

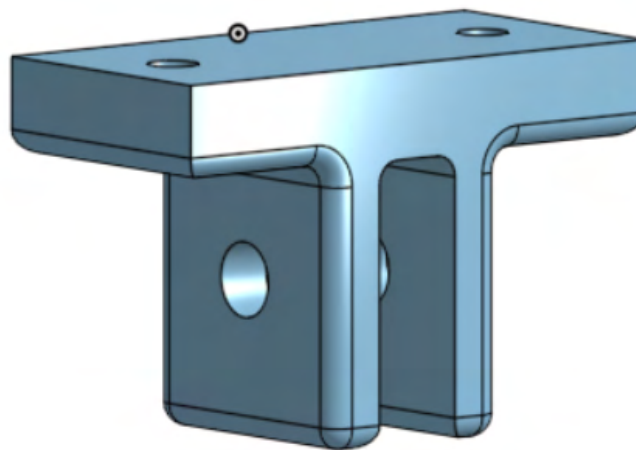


Figure #: Spring Mounts

This 3D printed spring mount serves as the primary support for a 3/8-inch diameter shoulder screw, which in turn connects to a spring with looped ends. Positioned beneath an adjustable 80/20 structure, it facilitates precise tensioning of the spring according to its displacement. This spring plays a pivotal role in preventing oscillations of the sprocket mechanism, particularly when encountering obstacles like speed bumps. The springs utilized to secure the linkage measure 4.5 inches in length, with an outer diameter of 0.5 inches and a wire diameter of 0.045 inches.

Drivetrain

The drivetrain consists of two drive wheels positioned coaxially at the center of the robot, along with two caster wheels on swivels, one at the front and one at the back of the robot. We

decided on this configuration because it requires only two motors, one on each drive wheel, as opposed to mecanum or triaxial drivebases while still allowing for rotation on a point. We are using two caster wheels rather than three for additional support, especially as the drive wheels will be positioned at the center line. This decision was made to minimize the moment the motors must overcome when turning.

The drive motors were attached to drive modules, pictured below. A majority of the power transmission parts were produced by GoBilda, as the selected motor was through the company as well and it was convenient to use a suite of products that work with the motor. Additionally, they were relatively cost effective.

The motor chosen was selected based on a worst case power requirement scenario, specifically:

- 35 lb max weight split evenly between the two drive wheels, the heaviest expected to stay below the 40 lb limit applied by OSHA.
- A 4 degree slope, slightly higher than the 3.8 degree limit for parking garages in the state of Massachusetts.
- A 3 ft/s max speed
- A 1 ft/s^2 max acceleration
- 40% efficiency, roughly average for brushed DC motors
- 2.8 inch wheels
- A factor of safety of 1.2, a standard value.

This resulted in a required torque of 0.7 pound feet at a speed of 245 rpm. In the future, the design will hopefully be compacted, which will reduce the motor requirements accordingly.

A brushed DC motor was selected. A brushless motor would have been ideal, as it would be quieter to minimize interference with the concrete sounding and have a higher efficiency, but the brushed motors were more cost effective for the torque required.

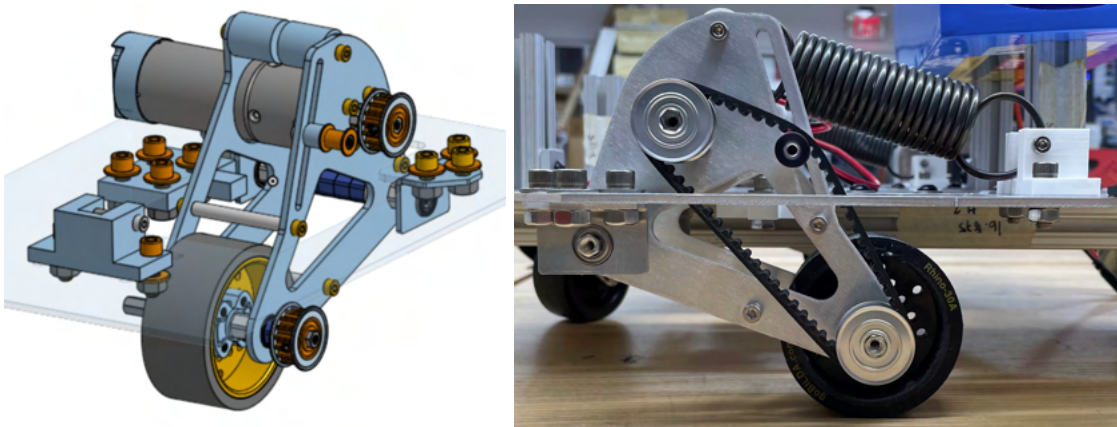


Figure X: Drive module assembly model (left) and prototype (right).

Drive Module Arms

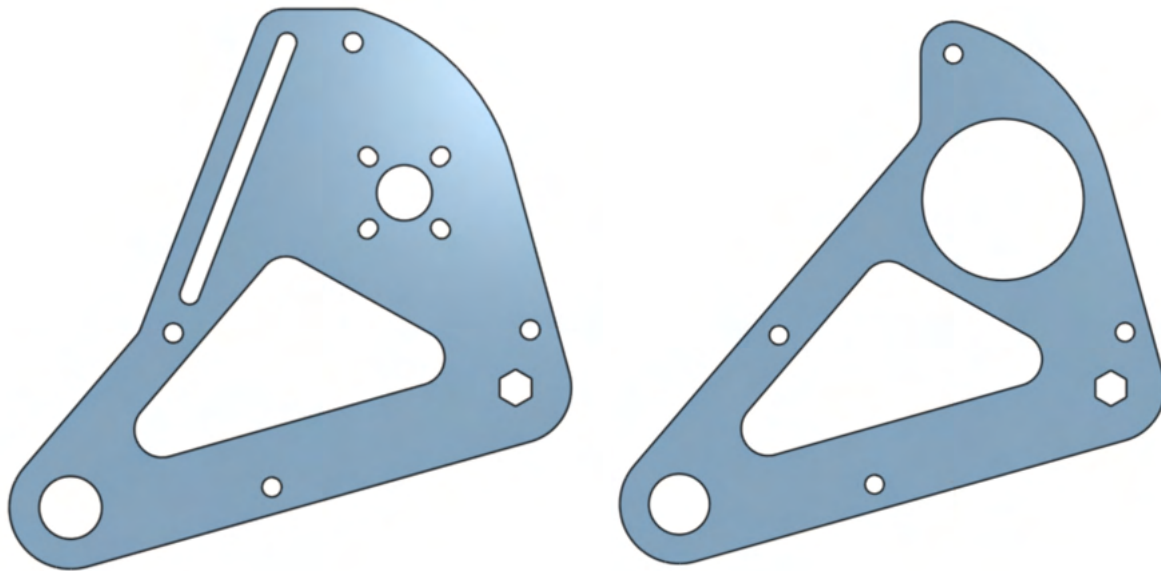


Figure X: Motor mounting arm (left) and passthrough arm (right).

The drive arms were designed in a triangular manner to allow for a more compact design in comparison to a linear alignment, such as in a c-channel. The arms were made from 1/8 inch 6061 aluminum, a strong and light alloy. The circular hole in the bottom left is designed to support a bearing through which the drive wheel was supported. An axle fits through the hexagonal hole in the bottom right. The drive module rotates around this axle, supported by mounting brackets attached to the chassis. The motor passes through the largest circular hole on the passthrough arm and is fastened to the inside of the motor mount arm, with the shaft passing through the hole. The motor mounting setup was inspired by GoBilda c-channel. The large triangular hole is intended for weight reduction, which was roughly arbitrary but qualitatively tested with 3D printed prototypes. The long slot on the motor mount arm is meant for tensioner attachment to allow for precise tensioning adjustment. Finally, the small holes fasten standoffs that attach the two arms. The topmost standoff supports the spring, used for supporting the robot's weight as well as suspension.

The size of the arms were based around the necessary positions of the axles and motor. The axles were positioned to attain a 3 inch ride height at neutral position with the axle mounted at a height that maintained the mounting brackets above the 80/20 to protect them from scraping the ground. The arms were designed after the bottom panel, so the horizontal length was based around the wheels being at the center with the motor mount being as far away as possible without reaching the slanted portion, maintaining maximum width. This was to balance the wheels being as far apart as possible to maximize stability, as well as keeping them as far away from the pivot so it could go over holes without approaching vertical.

Chassis Mounting

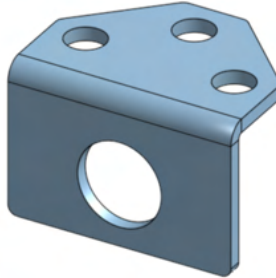


Figure X: Bracket for drive module to chassis mounting.

The drivetrain was mounted to the chassis through the use of the above mounting brackets. They were made from 0.09 inch 5052 aluminum sheet metal. This alloy was chosen because it was necessary for the part to be bent. SendCutSend, the company that fulfilled the order, didn't allow 6061 aluminum to be bent, so 5052 was the strongest suitable option. The thickness was chosen because it had a drastically smaller effective bend radius than other similar thicknesses, allowing for the holes to be located closer to the bend line.

The larger hole is meant for the placement of a bearing that will support an axle around which the drive arm rotates, and the smaller holes are meant for 0.25" fasteners. The triangle shape was chosen because it was important that the rotation of the part is limited even if a screw is loosened. The cutouts on the corners exist because the pieces closest to the edges of the base plate would extend past the bounds of the chassis, and making them all the same decreased costs and increased the ease of assembly. The length of the top portion was chosen to be as compact as possible while maintaining stability, and the height of the front portion was designed to be as tall as possible while remaining above the limit of the one inch 80/20 frame.

Spring Mounting and Selection

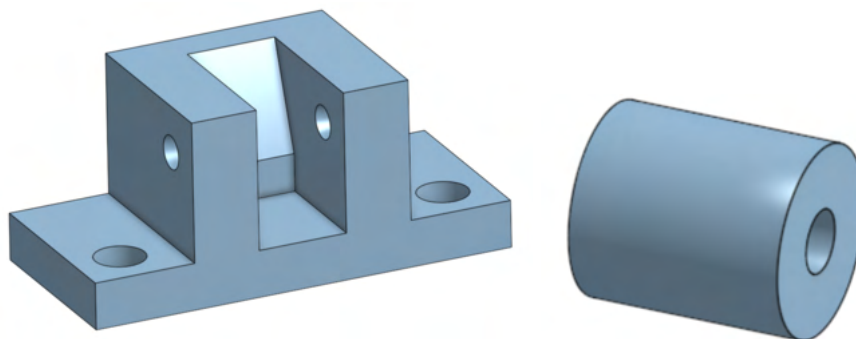


Figure X: Bracket for chassis to spring connection (left) and for location on drive module (right).

A spring was used to support the drive module and to provide suspension so drive wheels moving over bumps wouldn't interfere with the sensors. The spring was selected based on weight calculations at neutral ride height. It was assumed that $\frac{3}{8}$ ths of the total weight, 12 lbs, would

be supported by each wheel at ride height. Using lengths from the spring to the support, it was calculated that the spring would need to support 16 pounds at neutral. The spring was selected to maximize the stiffness at the fully extended position to ensure that the modules wouldn't bottom out. The mounting locations on the chassis had slots to attach, allowing for some tuning in unideal cases. It turned out that the position closest to the motor was ideal. The selected spring had a spring constant of 17.2 lb/in. While it supported the robot well, it ended up being too stiff to properly cushion small displacements. Additionally, the robot oscillates with uneven forces, so a damping system would be ideal in future iterations.

The bracket for attachment to the chassis was designed for the spring to go around a short standoff. Initial designs were too weak, so the bracket was made wider and a back was added. The slope in the back was added so that the spring could move backwards in the case that it was put in tension from above. The bracket to locate the spring on the drive module was rudimentary. They were long enough for half the distance between the drive arms minus half the diameter of the spring wire, so the spring could not move from side to side when between the locators. Additionally, the locators were made large enough to keep the spring vertical.

Hardstop

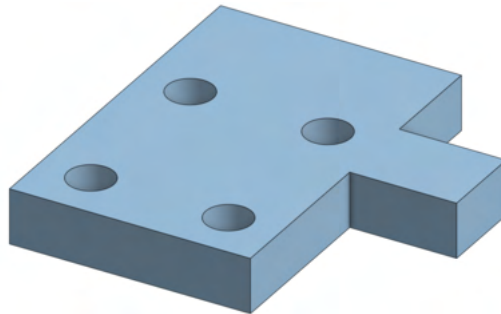


Figure X: Hardstop to limit drive module rotation.

One of the potential problems with the drive module was the possibility of bottoming out. The goal of the hardstop was to limit the rotation of the drive modules in order to control movement and avoid damage to parts. The hardstop was located below the chassis. The height of the hardstop corresponded with the parallel distance between the bottom of the weight cutout in the arms and the bottom of the chassis when at maximum rotation. The placement of the hardstop was determined by the point at the top of the weight cutout, so that it would lower as much as possible before being stopped. This prevented the motor hitting the chassis when the wheels were lowered. The lowering limitation was the most important, as the spring wouldn't be able to resist the compressive force it feels in that direction. The holes on the hardstop were off center to allow for assembly and limit interference with the motor. The hardstop was made out of plastic so that the hardstop would wear with interaction with the arm rather than the arm being damaged.

Tensioner

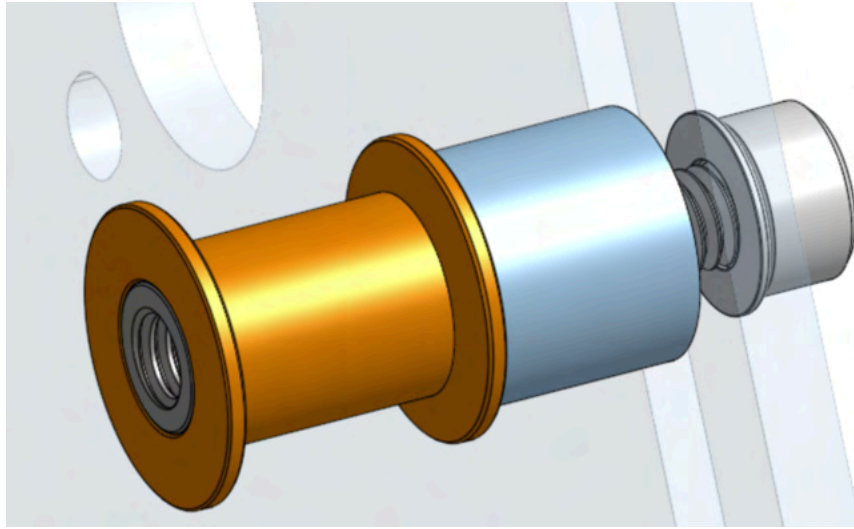


Figure X: Tensioner parts in assembly.

Power transmission from the motor to the drive wheel was achieved using a timing belt. The timing belt was intentionally purchased with a degree of slack. This required the use of a tensioner system to ensure proper meshing of the belt. This tensioner system consisted of a screw, 3D printed spacer, and an idler gear on the standoff. The 3D printed spacer was used to keep the idler at the same distance from the arm as the pulleys so it aligned with the belt. The system was designed to go inside the belt and pull upwards along the slot until the correct tension was achieved. It would have been ideal to push the belt inward so that the smooth face interfaced with the idler, but the belt was too loose and would touch the other side. One concern with this design is the possibility that the screw will come loose and slip down over time, something that could be solved with an insert or a spring offering tension upward.

Battery Mounting

The battery accounts for a large share of both the robot's weight and volume, so its location relative to the other subsystems is very important. The robot's battery weighs 5.85 lbs and has rectangular dimensions 4.72" x 2.75" x 10.03", as shown below.

For most of the project, robot layout plans had the battery squared away in the rear of the robot, just above the rear caster wheel, to leave plenty of space for the other subsystems. This spot was ideal for spacing because the rear of the robot didn't have any sensors or major mounting points, other than the drive train and sounder springs (Figure X). The mounting brackets were designed to clamp the battery in place so that there would be no room for the battery to move during travel or shake going over bumps. The bottom half of the clamp was to the 8020 uprights of the chassis and to the aluminum baseplate. The upper half of the clamp was to the lower half by partially threaded screws through heat-set inserts. The full-closed gap between the upper and lower pieces was smaller than the height of the battery so turning the screws continued

tightening the two halves around the battery. The friction between the clamps and the battery was enough to keep the battery from sliding side-to-side, but the clamps included hard stops just in case it wasn't. All four battery clamp components were 3D printed from PLA.

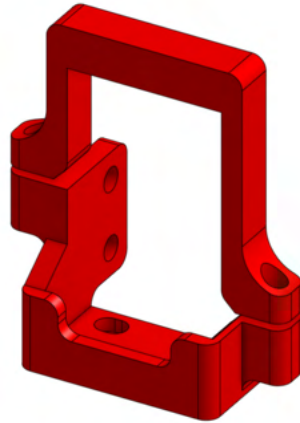


Figure X: CAD model of the left-side battery mounting brackets.

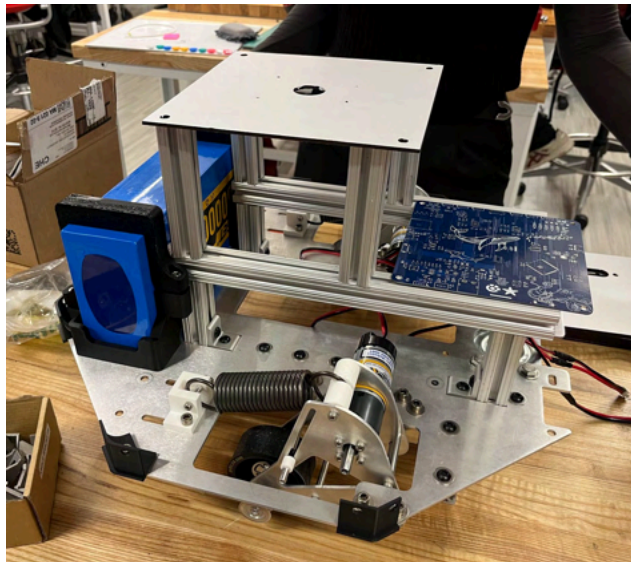


Figure X: Concept layout of the battery mounted with the chassis, top plate, and PCB.

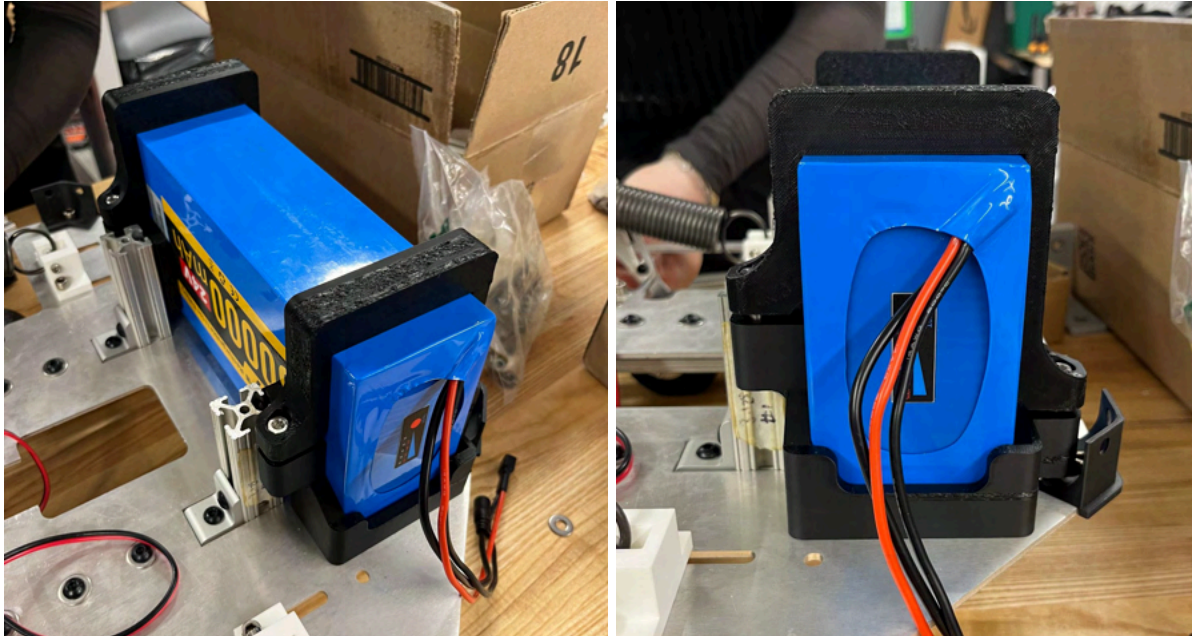


Figure X: Close-up views of the rear-mounted battery.

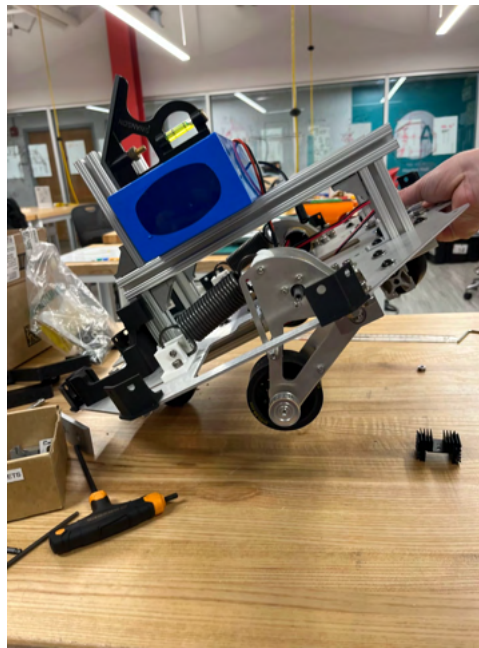
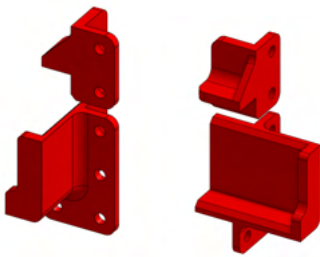
After mocking up this battery placement, shown in Figure X, several major problems were revealed. Being mounted directly over the rear caster wheel, most of the battery's weight was supported solely by the rear caster wheel (Figure X). Because the center of gravity of the robot was so far behind the drive wheels, none of the load was supported by the front caster wheel, and the front of the robot tilted upward so that the front caster wheel lost contact with the ground (Figure X). With this rear battery placement and the selected springs, there was not enough weight on the drivetrain to extend the springs and keep all four wheels on the ground.



The system shown in the free body diagram is statically indeterminate. The portion of the weight that is supported by the drivetrain springs depends on their stiffness: the stiffer the spring, the greater the share of the weight supported. Rather than respecify the springs, a new approach to

mounting the battery was devised. Similar to the previous iteration, the battery was mounted to the aluminum extrusion with flanged 3D printed components and drop-in T-nuts (Figure X).

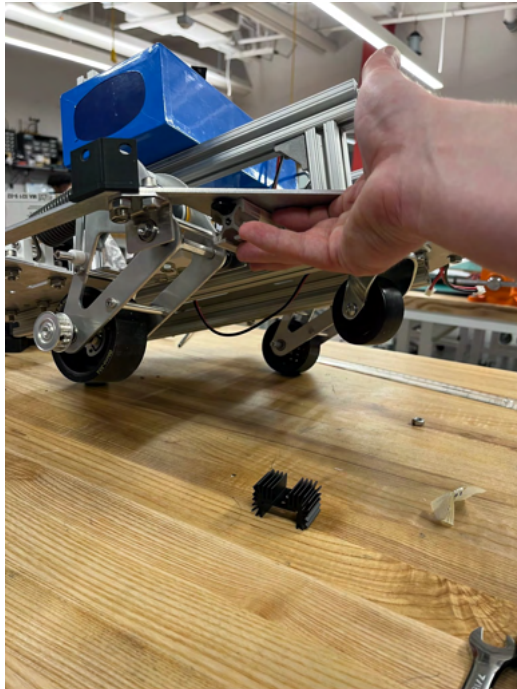
Different, however, was the placement and clamping of the battery. This time the battery was located as close to the center of the robot as possible to more equally distribute its weight among the front and rear caster wheels. The previous mounting brackets also clamped the battery more than was necessary so the new mounting brackets only rigidly locate the battery without clamping it.



To fit the battery in the middle of the robot's horizontal plane, the battery would only fit on the upper level of the robot's chassis, so a few concessions were made. The first is that lifting the battery up to the upper chassis raises the overall center of mass of the robot and increases the risk of it tipping over on a slope.

The tipping point of the robot was tested by resting the battery in its approximate position and lifting each edge of the chassis plate until the robot started to tip. In a tipping scenario, the robot would pivot around either one wheel or two adjacent wheels. All single wheels and all four

two-wheel combinations were tested (Figure X) and the tipping point of all of them exceeded the maximum expected slope in a parking garage.



Recommendations: Pay greater attention to CoM. Mount the battery so that a lot of the robot's weight is supported by the drive wheels.

Lower the center of mass so that the suspension doesn't rock back and forth

PCB Mounting

In the original rear battery placement, the central PCB was going to be mounted horizontally flat on the upper level (Figure X above). Moving the battery to the center of the upper level displaced the PCB. Requirements of PCB mounting:

- Allow access for a person to do maintenance and route cables.
- Allow space for cables to be plugged into the PCB ports and routed with a sufficient bend radius.
- Ports are positioned so that cables will reach their other connections (motors, sensors, etc).
- Electrically isolate the PCB from the chassis.

The PCB was relocated, for a number of reasons, to the rear of the robot where it would be mounted vertically.

- There were no gravity-dependent components on the PCB, so mounting it vertically did not seem to affect performance.
- There was ample space over the rear caster and a lot of mounting freedom afforded by the t-slot extrusion of the chassis.
- The rear bodywork panels have no embedded sensors so they can be easily removed to create access to the plugs on the board.

However, this PCB placement was not ideal and was not finalized before the PCB was sized and laid out. This caused conflicts when cables were routed because, now that the PCB is mounted vertically, the top and bottom edges of the board are constrained by the top body panel and the chassis baseplate, respectively.

-

There are cable ports on all four sides of the PCB so the the .'

The PCB was mounted so that the

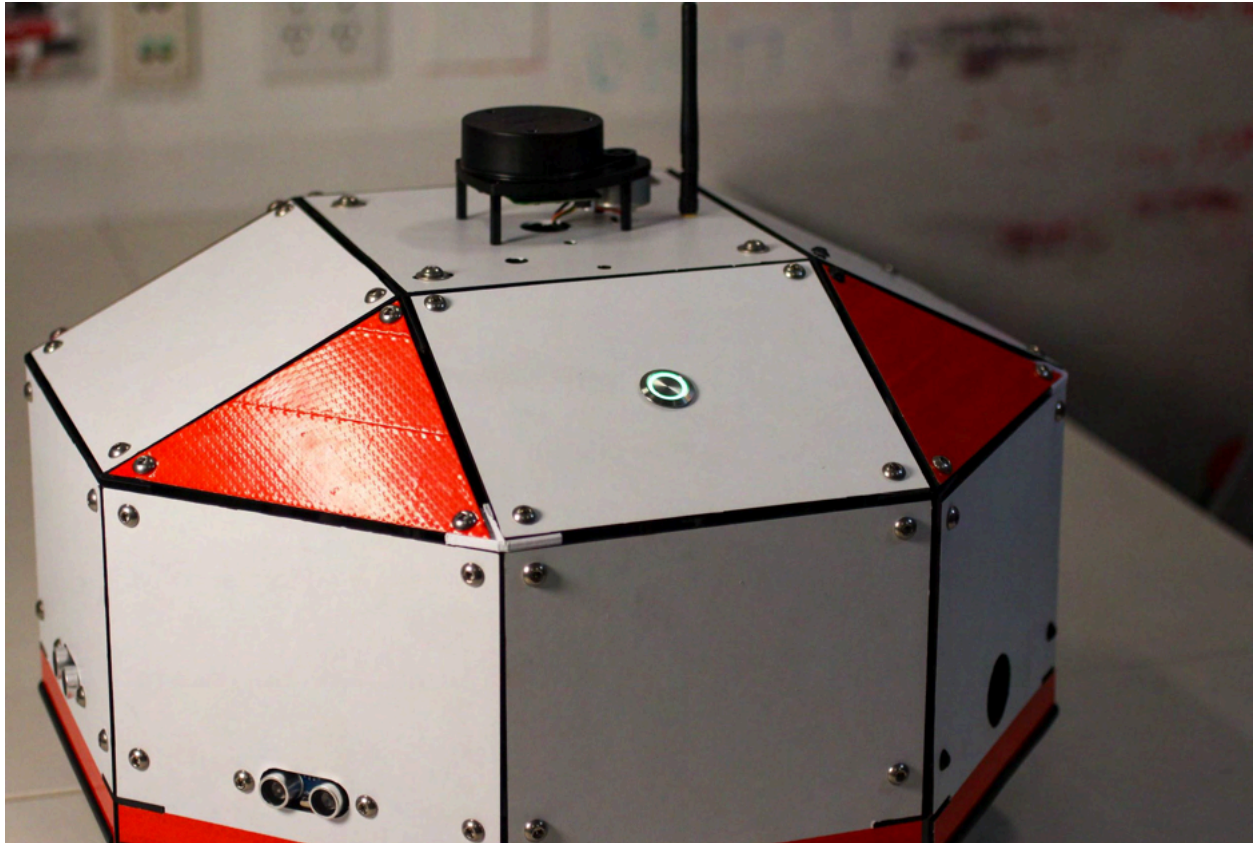
Problems:

We had to drill a hole in the chassis to plug a laptop into the board. That means the user has to flip the robot belly up to jack in.

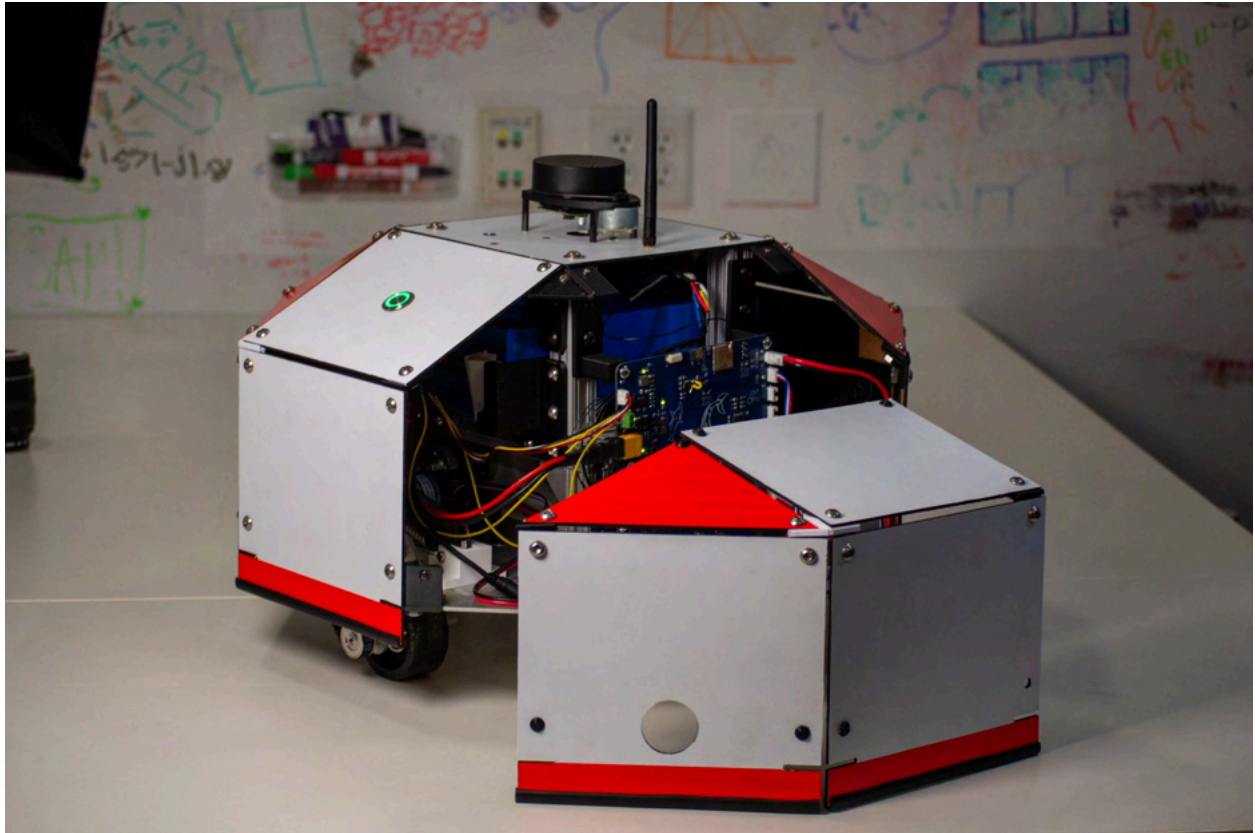
We didn't do any calculations to check if arcing could be an issue i.e. distance from board to chassis with operating voltage.

Body Work

For the body work of CSTAR for aesthetic and mobility purposes, an octagonal shape was selected, as a square would cause turn radius issues for the robot, and a circular shape would be infeasible given the times and materials available for our disposal. The body work's purpose for this project is to protect the inner components with shell, from outside interference. The outer body work is made out of Tempered Hardboard, specifically $\frac{1}{8}$ " Hardboard for it being lightweight because of restrictions of OSHA, and needed to be under 40lbs, finally being 28.4lbs. Hardboard was the final decision because of pricing constraints and weight constraints. For visibility purposes the hardboard is decaled with white reflective vinyl, and red reflective tape so that light from cars will reflect off to avoid damage to the robot; along with for aesthetic purposes. Visuals below:

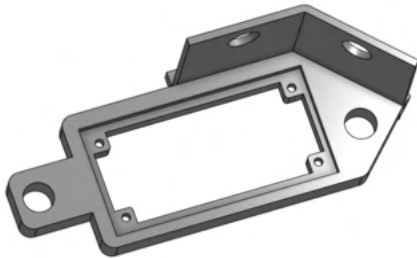


For internal access to the panel, magnetic panels on sections to the robot were implemented in front of the PCB and battery for assembly and testing convenience. The panel is a section of the rectangle section and triangle section side combined that clicks into place and is seamless with the rest of the body work. Visuals below in context:



For the shell because of its unique geometry it needed custom brackets for the geometry to be possible. With custom brackets for the top panels that slot into the 80-20 chassis supports, middle connecting brackets for the panel to panel connections, magnetic variant brackets, basic chassis connection for the panels, drivetrain variant brackets for the chassis to prevent interference, and magnetic insert for the panels and bracket all shown below:





For overall affordability, the shell's elements had a limited budget to ensure that the other elements could be purchased. However, this was not a limiting factor as the shell materials were designed to be lightweight. For another iteration, making the robot watertight would be ideal, but was out of scope for the amount of changes happening internally. The only items needed for the shell were the hardboard, 3D-printed brackets, fasteners, and the decal.

For next iterations of the project, making the panels out of a lightweight sheet metal might be a nice change for the project. Also having the brackets printed in a different material as well, SLA would probably be ideal. The shell could also be injection molded and have a plastic shell like a roomba in industry as well. Also there would also be better handles on the bottom of the chassis. The shell would ideally be water tight as well because of the injection molding.

User Interface/Interaction:

Because of time restraints for the project there wasn't a digital user interface for the user that wasn't the function code for the electrical and mechanical components. Physically for user interaction there was a button to turn on and off the robot, and the button lights in a ring light form around the button to communicate mode, which could also communicate system status as well.

Hardware Selection

Include the names and part numbers of the hardware involved in the assembly. This can be in the form of a table

This is an example. Do not feel obligated to use this format

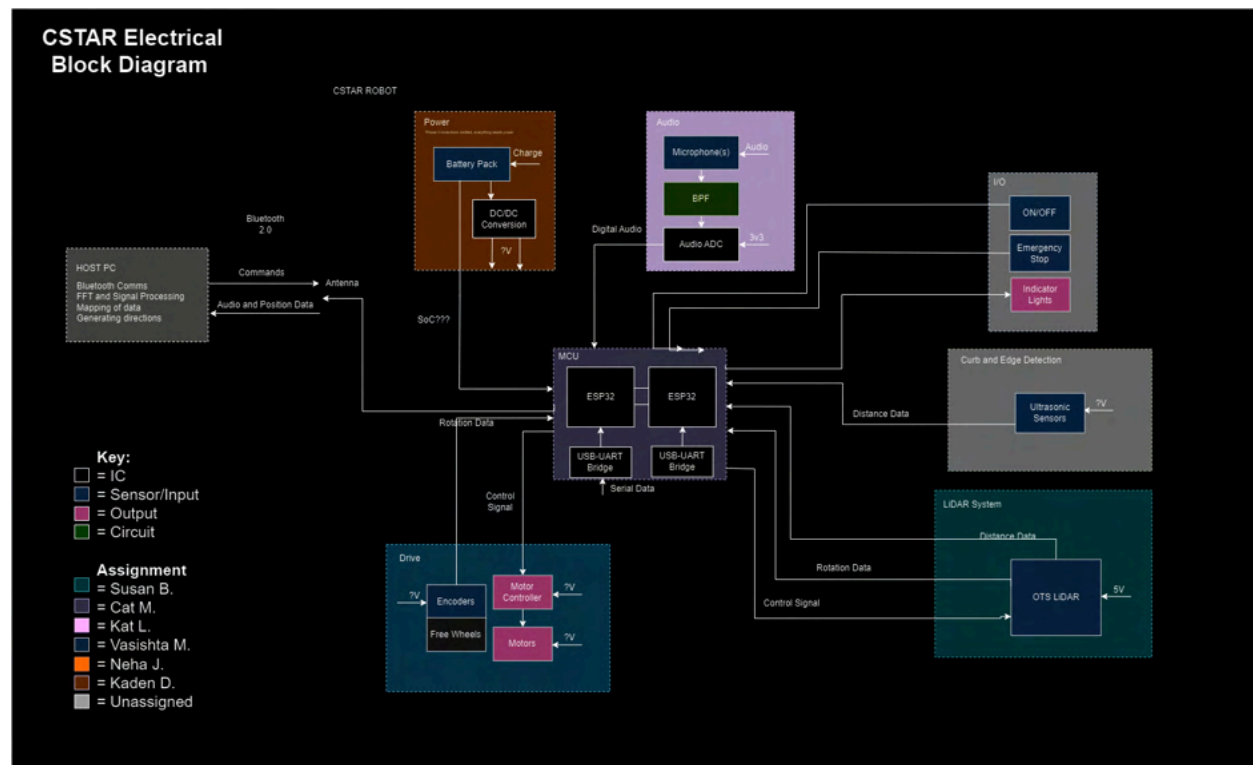


Manufacturer	Part Number	Quantity	Unit Cost	Total Cost
SLAMTEC	DFR0315	1	\$99	\$99
<i>Total</i>				\$

Electrical Design

Design Overview

- Flowchart of how components and subsystems interact
- Explanation of decisions made in using general technologies and concepts
- I/O of the electrical systems in relation to mechanical, software, etc.



The electrical system is designed to promote ease of recording audio, move the robot, and detect position of the robot. To accomplish this, 2 ESP32 Microcontrollers are used, one of which will communicate all data over bluetooth. An mic board was designed to record audio, and will connect to the main PCB.

Sensors

- What is each sensor's purpose?
- Where should each sensor be placed?
- Include a link to the spec sheet and call out basic information that's relevant to the application like accuracy. Be sure to relate information to the application

Ultrasonic Sensors for Ledge Detection

- Arduino Ultrasonic sensors use SONAR to determine the distance of an object (just like the bats do).
- We initially wanted to implement our custom ultrasonic sensor, this would allow us to have more control over mounting the ultrasonic sensor on the robot and be able to make improvements to the ultrasonic schematic, making it have a larger range and more accurate than the Arduino sensor. Unfortunately, there was not enough documentation on designing an ultrasonic sensor and we decided to not risk creating our own ultrasonic that would not work for the final project. The regular Arduino ultrasonic was used but as a goal for if this project continues, making our own ultrasonic sensor would be the goal.
- Why are we using ultrasonic sensors for this?
 - Good for detecting objects and edges in diverse settings
 - Accurate distance measurement to objects
 - Effective obstacle avoidance
 - excellent non-contact range detection → up to 13 feet!
 - high accuracy and stable readings → NOT affected by sunlight or black material

Features	
Max range	200 cm
Min range	3 cm
Effectual Angle	<15°
Resolution	1 cm
Power supply	+5V DC
Global Current Consumption	15 mA
Ultrasonic Frequency	40k Hz
Dimensions	43x20x15 mm

Table 1: This is a table that I made. It's pretty cool. This is the stuff I tested using the ultrasonics.
(NEEDS a better caption)

Circuit Board

- Schematics
- Any input / output and where it connects to sensors, mechanical devices, etc.
- Images to show relative size and placement within the overall system

M.C.U

The microcontroller chosen for this project needed to have at least two UART connections, be capable of I2S and Bluetooth, and have a surplus of GPIO pins capable of PWM. This led us to choose the ESP32-WROOM-32UE. It's compatible with Arduino IDE, has multiple UART connections, and has enough pins to support the amount of pins needed for ultrasonics, motors, and LIDAR.

Level Shifters

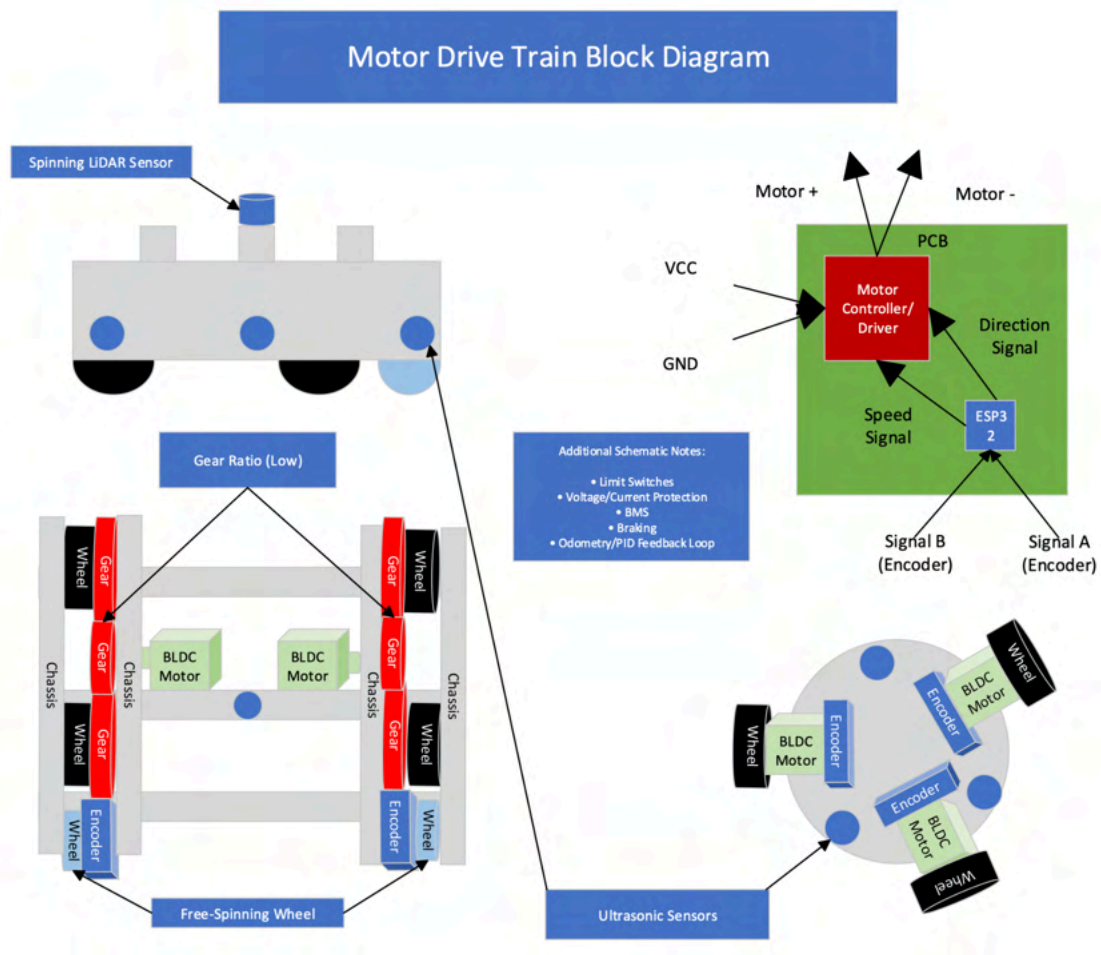
The ultrasonics signal needs 5V and the ESP32 can only offer 3.3V. The level shifters were used to convert the 3.3V trig signal to 5V, and convert the echo from 5V from the ultrasonic to 3.3V for the ESP32.

GPIO Expander

There were not enough pins on ESP2 for the motors and we needed all motor pins on the same ESP. A GPIO expander was chosen so that we could put digital signals for the motor on the expander.

Drivetrain

The drivetrain consists of two brushed DC motors controlled by a motor driver IC. The brushed DC motors also have optical encoders that provide feedback to the ESP32-WROOM-32UE microcontroller for the odometry, PID, and SLAM algorithms. The encoder code was written using the ESP32 Library found online. The code would trigger an interrupt every time channel A on the encoder was triggered. The phase shift of channel B in relation to channel A would determine a positive or negative increment of the position counter. This encoderPosition variable would then be used in further calculations for our odometry and P.I.D algorithms that would allow us to move towards a certain distance, and orient the robot to any degree we would like. Some issues that we encountered while



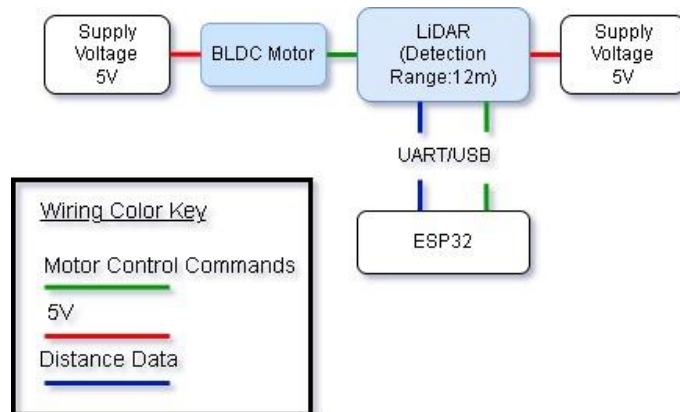
Lidar Sensor

The Lidar sensor has been selected with the purpose of object detection and avoidance, to act as the “eyes” of the robot.

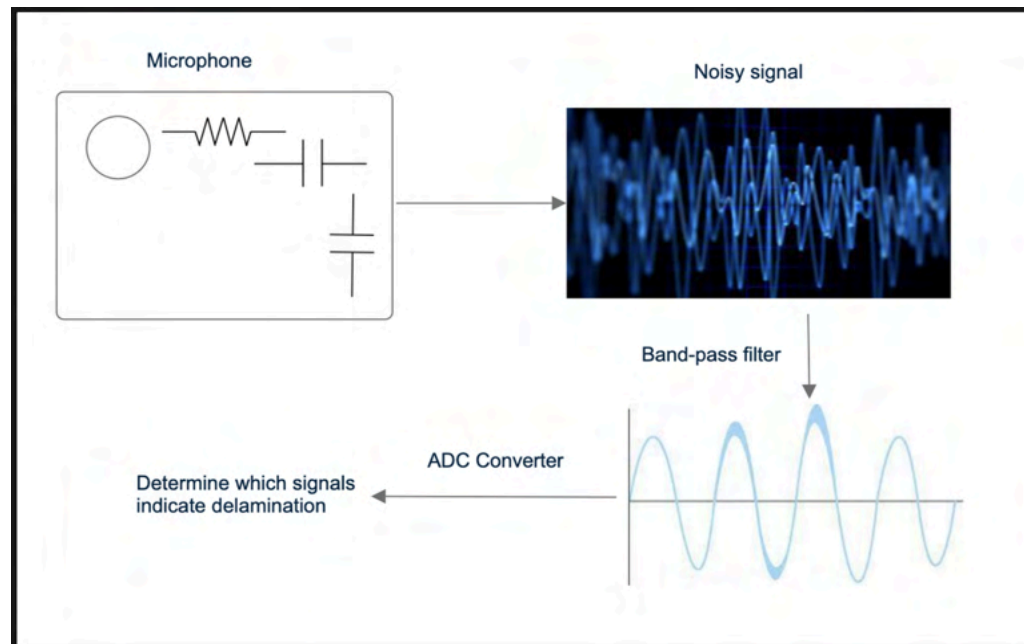
The Lidar selected was chosen due to the scanning detection range, the integrated BLDC motor, the communication interface, and the SDK package available. Due to client requirements a detection range higher than 10 meters was desired for object detection around a complex area, such as a garage.

In order to start with the testing of the Lidar selected, a python script was utilized. Python contains a library previously developed that contains the necessary function for the Lidar to start scanning, stop scanning, get the Lidar manufacturing information, and the health of the Lidar to see if it is in a good state or not. With the use of the RPLidar library, multiple python scripts were developed to obtain the Lidar details, including the status, the date and time, the health status, if

the device was found or not. After making sure that the data was being sent correctly, with angle and distance measurement in mm,



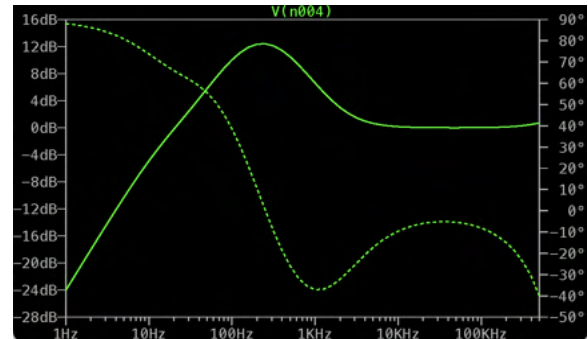
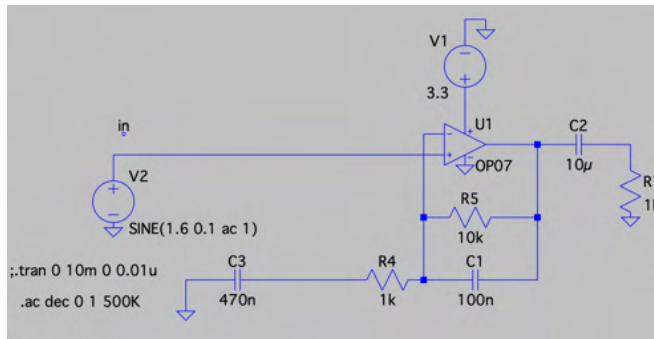
Audio System



The purpose of the audio system is to cancel outside noise and digitally record audio to determine which frequencies indicate delaminated concrete.

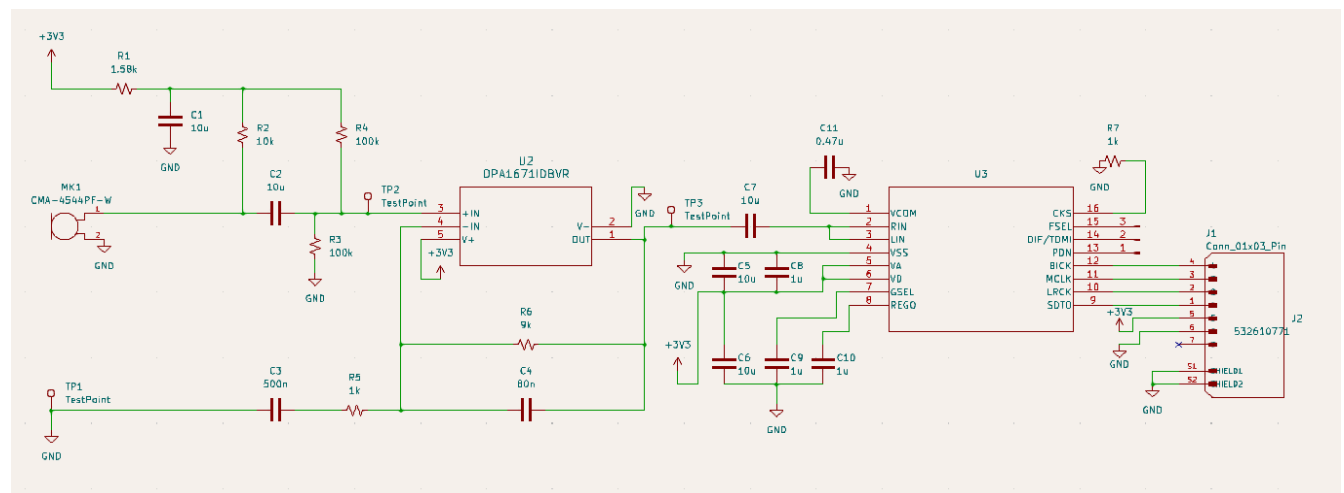
The audio system consists of two microphones: one reference microphone that is close to the sprockets to capture the concrete sound (desired sound), as well as one noise microphone that is mounted to the chassis to capture environmental noise (extra noise). The noises from the

microphone will go through a bandpass filter to attenuate frequencies from 100 Hz to 1 KHz. The signal will eventually go through an ADC to send digital signals to the host PC. From there, an FFT can be done in order to determine which signals indicate delamination. This microphone system uses I2S communication.

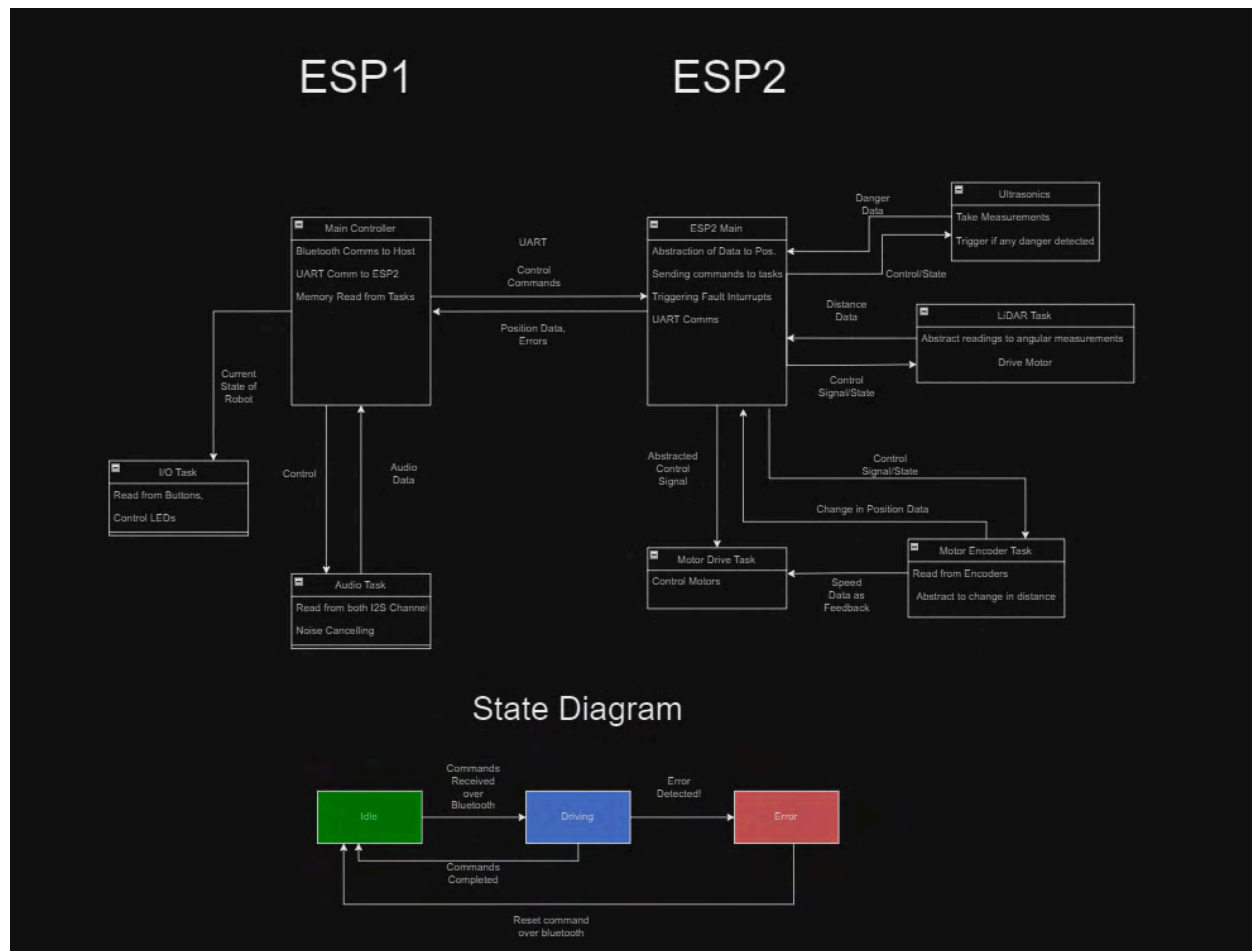


The microphone pre-amplifier circuit was simulated through LTSpice, which illustrates the bandpass filter being cut off at around 100 - 1000 Hz.

Schematic Design



The circuit is displayed above, where the output of the ADC is connected to a connector, which will be connected to the main board PCB. The FSEL, DIF, and PDN pins are meant to be connected to a pin header. From the main PCB pin, the audio data can be streamed through bluetooth.



Firmware Outline



As the project was focused mainly on hardware proof-of-concepts, the firmware and software portions of the project still have a ways to go. The main portion of the software was designed using FreeRTOS. Each of the main tasks for each subsystem was designed as a separate task for the RTOS to process. These tasks used FIFOs in shared memory to communicate. On the first ESP, one task is used to collect audio data, one to stream to bluetooth, one to communicate over UART and one to control the RGB button. On the second ESP, one task is used to control the motors, one for LiDAR, one for ultrasonics, one to do sensor fusion between encoders and LiDAR, and one to communicate over UART. The current state of the firmware has most functionality, but our LiDAR, encoder, and ultrasonic sensors firmware is unfinished.

Software Outline

As a basic proof of concept for the software portion of the project, we used a python app which will connect over bluetooth to the robot. After connecting, the robot will stream audio and position data over bluetooth, and the python program will stream commands to the robot. As audio data is received from the robot, it is processed by taking the FFT and checking the magnitude of the resulting signal in the 300-500Hz range. The required libraries for this project are:

1. Multiprocessing
2. Custom Tkinter
3. Numpy
4. Scipy
5. Pyserial
6. Matplotlib



Next Steps

Mechanical Design

Drivetrain

- Add dampers to limit oscillation
- Spec smaller motor for updated robot size
- Add suspension to caster wheels
- Tune spring so it better handles bumps of all sizes
- Reduce cost of drive arms, perhaps through more proprietary parts
- Run driving tests both in a parking garage and in controlled extreme conditions: increasing bump size, increasing ramps angle, different ground surface finishes/materials, etc.
- Remount the PCB somewhere else

Electrical Design

Software Design

Define exactly where the project stands

What works? What doesn't?

What are the existing projects / shortcomings?

What do you recommend doing next?

Getting quotes from manufacturers?

Gathering user feedback

Next Steps

Appendix

Audio and Bluetooth code:

```
#include <Arduino.h>
#include <BluetoothSerial.h>
#include <SoftwareSerial.h>

#include "soc/rtc_cntl_reg.h"
#include "soc/rtc.h"
#include "soc/syscon_reg.h"
#include "soc/rtc_cntl_struct.h"
#include "driver/i2s.h"

// Pin values from MCU Schematic
#define red_pin 21
#define green_pin 23
#define blue_pin 33

#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please run `make menuconfig` to and enable it
#endif

#define RX 12
#define TX 27
SoftwareSerial SwSerial(RX, TX); // 14, 12 on pins
BluetoothSerial SerialBT;
int msg;

void setGreen() {    // Running = Green
    digitalWrite(red_pin, LOW);
    digitalWrite(green_pin, HIGH);
    digitalWrite(blue_pin, LOW);
}

// audio port
static const i2s_port_t i2s_num = I2S_NUM_0;
```



```
static const i2s_config_t i2s_config = {
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
    .sample_rate = 44100,
    .bits_per_sample = I2S_BITS_PER_SAMPLE_24BIT,
    .channel_format = I2S_CHANNEL_FMT_RIGHT_LEFT,
    .communication_format = (i2s_comm_format_t)(I2S_COMM_FORMAT_I2S |
I2S_COMM_FORMAT_I2S_MSB),
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,    // high interrupt priority
    .dma_buf_count = 8,                        // 8 buffers
    .dma_buf_len = 1024,                      // 1K per buffer, so 8K of buffer space
    .use_apll=0,
    .tx_desc_auto_clear= true,
    .fixed_mclk=-1
};

static const i2s_pin_config_t pin_config = {
    .bck_io_num = 19,                        // The bit clock connection, goes to pin 27 of ESP32 1 BCLK
    .ws_io_num = 14,                        // Word select, also known as word select or left right clock 26
2 LRCK
    .data_out_num = I2S_PIN_NO_CHANGE,        // Data out from the ESP32, connect to DIN
on 38357A 25 3 I2S_PIN_NO_CHANGE
    .data_in_num = 26                        // we are not interested in I2S data into the ESP32
I2S_PIN_NO_CHANGE SDTO 25
};

void setup() {

    // define all pins
    // SerialBT.begin("ESP32testing"); //Bluetooth device name
    SwSerial.begin(38400);
    Serial.begin(115200);
    // SerialBT.begin("ESP32testing"); //Bluetooth device name
    // Serial.println("The device started, now you can pair it with bluetooth!");
    pinMode(RX, INPUT);
    pinMode(TX, OUTPUT);

    // set up pin colors
    pinMode(blue_pin, OUTPUT);
}
```



```
pinMode(green_pin, OUTPUT);
pinMode(red_pin, OUTPUT);

// begin bluetooth
SerialBT.begin("CSTAR Robot2"); //Bluetooth device name

// set up i2s
i2s_driver_install(i2s_num, &i2s_config, 0, NULL);    // ESP32 will allocated resources to run I2S
i2s_set_pin(i2s_num, &pin_config);

const int ledPin = 18; // GPIO pin connected to the LED or other PWM device
const int ledChannel = 0; // LEDC channel (0-15)

// Setup PWM properties
const int freq = 40000; // Frequency in Hertz (40 kHz is high but still common)
const int resolution = 8; // Resolution in bits (1-16, 8 bits gives 0-255 range for duty cycle)

// Configure LED PWM functionalitites
ledcSetup(ledChannel, freq, resolution);

// Attach the channel to the GPIO to be controlled
ledcAttachPin(ledPin, ledChannel);

// Set the duty cycle (0-255) due to 8-bit resolution
ledcWrite(ledChannel, 127); // 50% duty cycle

}

void loop() {
  // put your main code here, to run repeatedly:

  // set led green
  setGreen();

  // read from bluetooth
  if (SerialBT.available()) {
    msg = SerialBT.read();
    Serial.write(msg);
    SwSerial.write(msg);
  }
}
```



```
}
```

```
uint32_t audioData; // Assuming you're reading 24-bit samples
size_t bytesRead;
// Read data from the I2S interface
analogWrite(18, 128);
i2s_read(i2s_num, &audioData, sizeof(audioData), &bytesRead, portMAX_DELAY);

// Print the number of bytes read
// transmit audio data
if (audioData != 0) {
    Serial.println(audioData);
    SerialBT.println(audioData);
}
//SerialBT.println(audioData);
}
```